

Scaling Laws for Large Language Models

A Comprehensive Study Guide - Part 2

Lecture by Yann Dubois
Stanford CS229: Machine Learning
August 13th, 2024

Compiled: February 1, 2026

Abstract

This document presents comprehensive study notes on **scaling laws** for Large Language Models (LLMs), covering the empirical relationships between model size, training data, compute resources, and performance. Scaling laws provide a predictive framework for understanding how LLM performance improves with scale, enabling optimal resource allocation and architecture decisions. Topics include the fundamental power-law relationships, the Chinchilla scaling law for optimal compute allocation, practical applications for hyperparameter tuning, architectural comparisons, and real-world cost analyses. This guide synthesizes lecture content from Stanford's CS229 course with detailed visual representations and mathematical formulations.

Contents

1	Introduction to Scaling Laws	4
1.1	The Scaling Phenomenon	4
1.2	Breaking the Overfitting Paradigm	4
1.3	The Core Question	4
2	Fundamental Scaling Laws	5
2.1	The Power-Law Relationships	5
2.1.1	Mathematical Formulation	5
2.2	Visual Representation	5
2.3	Key Properties of Scaling Laws	6
2.4	Understanding the Metrics	6
2.4.1	Test Loss vs. Perplexity	6
2.4.2	Compute, Data, and Parameters Relationship	7
2.5	Empirical Validation	7
2.6	Current Status: 2020-2024	7
3	Practical Applications of Scaling Laws	8
3.1	Hyperparameter Tuning: Old vs. New Paradigm	8
3.1.1	Old Pipeline (Pre-Scaling Laws)	8
3.1.2	New Pipeline (With Scaling Laws)	8
3.2	Scaling Recipes	9
4	Architectural Comparisons via Scaling Laws	9
4.1	Case Study: Transformers vs. LSTMs	9
4.1.1	Methodology	9
4.1.2	Results Visualization	10
4.2	Understanding Scaling Curves	10
4.3	Sensitivity to Architectural Changes	11
5	Optimal Resource Allocation: Chinchilla	11
5.1	The Fundamental Trade-off	11
5.2	The Chinchilla Methodology	12
5.2.1	Experimental Design	12
5.3	Visual Representation	12
5.4	Key Findings	12
5.5	Mathematical Relationships	13
5.6	Implications for Model Training	13
6	Broader Applications of Scaling Laws	14
6.1	Questions Answerable with Scaling Laws	14
6.2	Data-Related Decisions	14
6.2.1	Data Repetition and Multiple Epochs	14
6.2.2	Data Mixture Weighting	15
6.3	Algorithmic Decisions	15
6.3.1	Width vs. Depth	15
6.4	Strategic Planning	16
6.4.1	Predicting Future Performance	16

7	The Bitter Lesson	16
7.1	Richard Sutton’s Insight	16
7.2	Application to LLMs	17
7.3	Implications for Research	17
7.4	Philosophical Implications	18
8	Real-World Cost Analysis: LLaMA 3 400B	19
8.1	Model Specifications	19
8.2	FLOP Calculation	19
8.2.1	LLaMA 3 Computation	20
8.3	Time and Compute Requirements	20
8.3.1	GPU-Hours Calculation	20
8.3.2	Wall-Clock Time	21
8.4	Cost Breakdown	21
8.4.1	Compute Costs	21
8.4.2	Personnel Costs	22
8.4.3	Total Cost	22
8.5	Environmental Impact	23
8.5.1	Carbon Emissions Calculation	23
8.6	The Next Generation	24
9	Summary and Key Takeaways	24
9.1	Core Principles	24
9.2	Practical Guidelines	25
9.3	Strategic Insights	25
9.4	Open Questions	25
9.5	Looking Forward	26
A	Mathematical Reference	26
A.1	Key Formulas	26
A.1.1	Scaling Law Power Laws	26
A.1.2	Chinchilla Optimal Scaling	27
A.1.3	FLOP Calculation	27
A.1.4	Time and Cost	27
A.2	Notation Guide	27
B	Additional Resources	27
B.1	Seminal Papers	27
B.2	Technical Reports	28
B.3	Online Resources	28
C	Glossary	28

1 Introduction to Scaling Laws

1.1 The Scaling Phenomenon

Scaling Laws

Scaling laws are empirically observed power-law relationships that describe how the performance (typically measured by loss) of language models improves as a function of:

- Model size (number of parameters)
- Dataset size (number of tokens)
- Compute budget (FLOPs)

Discovery and Impact

Around 2020, researchers empirically demonstrated and theoretically characterized a remarkable phenomenon: *More data and larger models consistently lead to better performance*. This observation has fundamentally reshaped how we approach LLM development.

1.2 Breaking the Overfitting Paradigm

Paradigm Shift

Traditional machine learning wisdom teaches that larger models lead to **overfitting**. However, LLMs exhibit fundamentally different behavior:

Larger models $\Rightarrow$ Better performance (no overfitting)

This counterintuitive finding took considerable time for the community to accept, as it contradicts standard ML curriculum teachings.

Academic Note

While overfitting doesn't occur with LLMs at current scales, *for academic exams and traditional ML contexts, overfitting still exists and should be considered!*

1.3 The Core Question

Predictive Power

Given that more data and larger models always improve performance, the fundamental question becomes:

*Can we predict how much better performance will be
if we increase data and model size?*

Surprising answer: Yes! And the predictions work remarkably well.

2 Fundamental Scaling Laws

2.1 The Power-Law Relationships

The seminal work by Kaplan et al. (2020) from OpenAI demonstrated three fundamental power-law relationships.

2.1.1 Mathematical Formulation

Power-Law Scaling

The test loss L scales with three key factors according to power laws:

$$\text{Compute: } L = \left(\frac{C_{\text{crit}}}{C} \right)^{\alpha_C} \times 10^{\beta_C + \epsilon_C^2} \quad (1)$$

$$\text{Dataset Size: } L = \left(\frac{D_0}{D} \right)^{\alpha_D} \times 10^{\beta_D + \epsilon_D^2} \quad (2)$$

$$\text{Parameters: } L = \left(\frac{N_0}{N} \right)^{\alpha_N} \times 10^{\beta_N + \epsilon_N^2} \quad (3)$$

where:

- C = compute (PetaFLOP-days, non-embedding)
- D = dataset size (number of tokens)
- N = number of parameters (non-embedding)
- α = scaling exponent (slope in log-log space)
- β = intercept term
- ϵ = residual error

2.2 Visual Representation

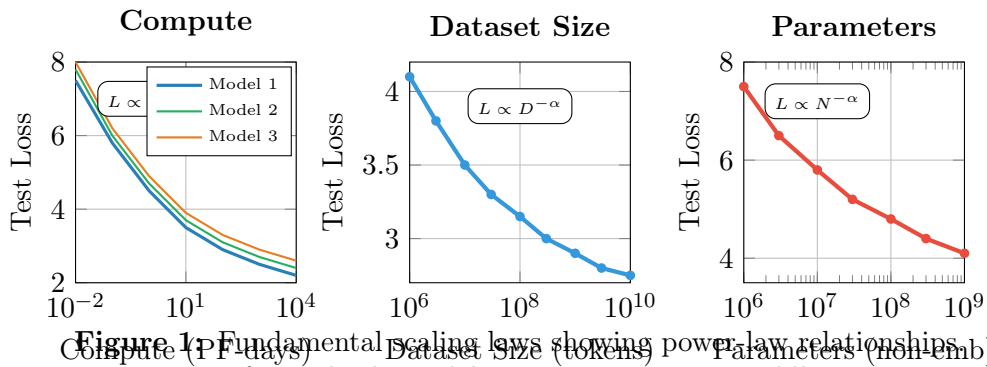


Figure 1 Fundamental scaling laws showing power-law relationships.
 Left: Multiple model sizes vs. compute. Middle:
 Dataset size vs. loss. Right: Parameters vs. loss.
All relationships are linear in log-log space, indicating power-law scaling.

2.3 Key Properties of Scaling Laws

Log-Log Linearity

When plotted on **log-log scale**, the relationship between compute/data/parameters and loss is **linear**. This means:

$$\log L = -\alpha \log X + \beta$$

where X is compute, data, or parameters. This linear relationship in log-space makes predictions remarkably tractable.

Predictive Power

The linearity of scaling laws in log-log space enables:

1. **Forecasting:** Predict performance in 2-3 years based on expected compute growth
2. **Resource planning:** Determine optimal allocation of computational resources
3. **Architecture selection:** Compare different architectures at scale
4. **Investment decisions:** Evaluate ROI of additional compute/data

2.4 Understanding the Metrics

2.4.1 Test Loss vs. Perplexity

Loss in Scaling Laws

The loss L used in scaling laws is the **cross-entropy loss** on validation data:

$$L = -\frac{1}{N} \sum_{i=1}^N \log p(x_i | x_{1:i-1})$$

This is related to perplexity by:

$$\text{PPL} = 2^L \quad \Rightarrow \quad L = \log_2(\text{PPL})$$

Why Use Loss Instead of Perplexity?

In scaling law research, loss is preferred over perplexity because:

- More natural for power-law formulations
- Easier to work with mathematically
- Direct relationship to the training objective
- Simpler linear relationships in log-space

2.4.2 Compute, Data, and Parameters Relationship

Interrelated Factors

The three scaling factors are **not independent**:

$$\text{Compute (FLOPs)} \approx 6 \times \text{Parameters} \times \text{Tokens}$$

Increasing parameters requires more compute *per token*. To maintain efficiency, you should also increase the number of training tokens. This interdependence is crucial for optimal training.

2.5 Empirical Validation

Robustness of Scaling Laws

Scaling laws have been validated across:

- Multiple architectures (Transformers, LSTMs, etc.)
- Different domains (text, code, multilingual)
- Various model sizes (10^6 to $10^{11}+$ parameters)
- Different organizations (OpenAI, Google, Meta, Anthropic)

The relationships hold remarkably consistently, suggesting they capture fundamental properties of neural language modeling.

2.6 Current Status: 2020-2024

Continued Validity

As of 2024, scaling laws **continue to hold**:

- No evidence of plateauing yet
- GPT-4, LLaMA 3, Claude 3 all follow predicted trends
- Industry continues to rely on scaling law predictions

Will they plateau? Probably eventually, but:

- No empirical evidence of when
- Mathematically, log-scale relationships don't *need* to plateau
- Most researchers expect eventual plateauing, timeline unknown

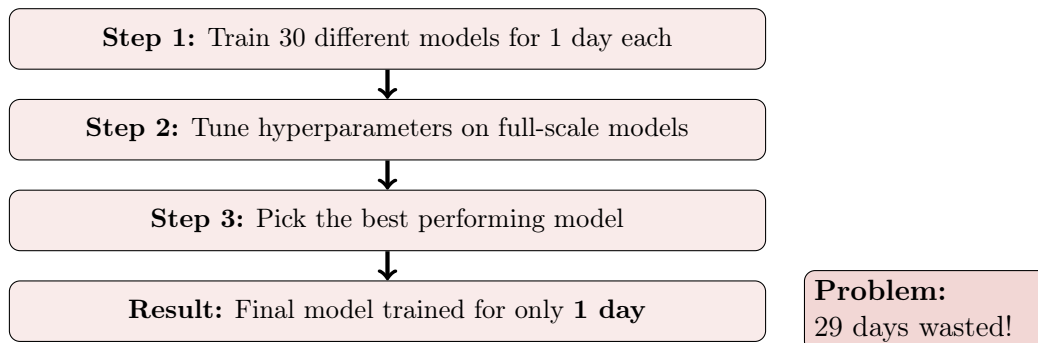
3 Practical Applications of Scaling Laws

3.1 Hyperparameter Tuning: Old vs. New Paradigm

Scenario: 10

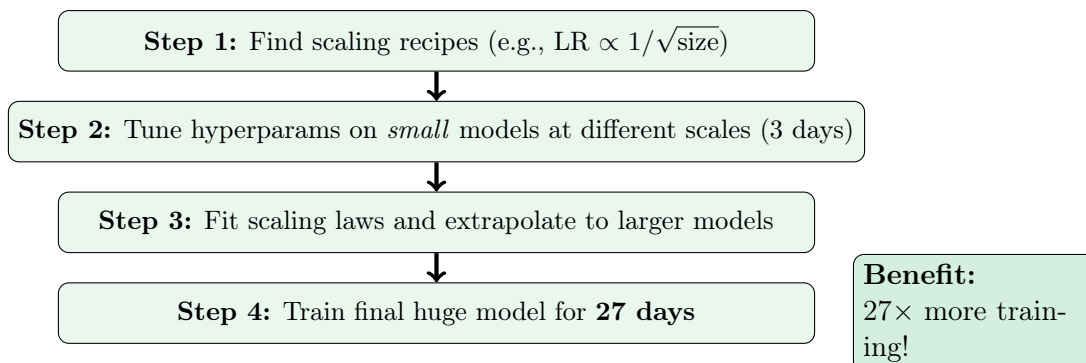
You have access to 10,000 GPUs for 30 days. What model should you train? How do you determine the optimal architecture, learning rate, and other hyperparameters?

3.1.1 Old Pipeline (Pre-Scaling Laws)



Major inefficiency: The final production model only benefits from 1/30th of the available compute time!

3.1.2 New Pipeline (With Scaling Laws)



Efficiency Gain

By using scaling laws:

- Final model receives **27 days** of training (vs. 1 day)
- Hyperparameters validated across multiple scales
- More reliable predictions of final performance
- Vastly more efficient use of computational resources

3.2 Scaling Recipes

Scaling Recipe

A **scaling recipe** is a set of rules that describe how hyperparameters should change as model size increases. Common examples:

- **Learning rate:** $\text{LR} \propto \frac{1}{\sqrt{N}}$ where N is model size
- **Batch size:** Increase proportionally with model size
- **Warmup steps:** Scale with dataset size
- **Weight decay:** Decrease with model size

Developing Scaling Recipes

Process for creating scaling recipes:

1. Train models at 3-5 different scales (e.g., 100M, 300M, 1B, 3B parameters)
2. For each scale, sweep hyperparameters
3. Identify patterns in optimal hyperparameters vs. scale
4. Fit mathematical relationships (often power laws)
5. Validate extrapolations with held-out scales

4 Architectural Comparisons via Scaling Laws

4.1 Case Study: Transformers vs. LSTMs

Architecture Decision

Question: You have 10,000 GPUs. Should you build a Transformer-based model or an LSTM-based model?

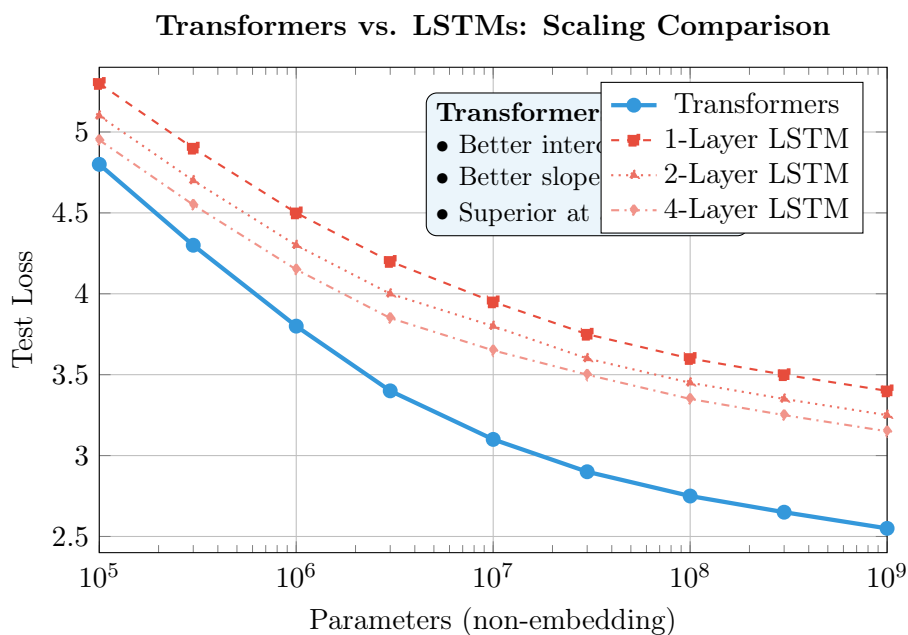
Traditional approach: Train one of each and compare.

Scaling law approach: Train multiple sizes of each, fit scaling laws, and predict performance at the target scale.

4.1.1 Methodology

1. Train Transformers at scales: 10^6 , 10^7 , 10^8 , 10^9 parameters
2. Train LSTMs (1-layer, 2-layer, 4-layer) at same scales
3. Plot test loss vs. parameters on log-log scale
4. Fit linear trends (power laws)
5. Extrapolate to 10^{10} parameters

4.1.2 Results Visualization



4.2 Understanding Scaling Curves

Two Critical Factors

When comparing architectures via scaling laws, examine:

1. Intercept (Initial Performance):

- Where does the curve start?
- Better intercept = better small-scale performance

2. Slope (Scaling Rate):

- How steeply does performance improve with scale?
- Steeper slope = better utilization of additional resources
- **Slope often matters more long-term!**

Transformers vs. LSTMs: Transformers dominate on *both* factors—better starting point *and* better scaling.

Possible Outcomes

When comparing architectures A and B:

- **Case 1:** A better intercept and slope $\Rightarrow$ A dominates (Transformers vs. LSTMs)
- **Case 2:** A better intercept, B better slope $\Rightarrow$ B eventually wins at scale
- **Case 3:** B better intercept, A better slope $\Rightarrow$ Crossover point exists
- **Case 4:** B better intercept and slope $\Rightarrow$ B dominates

Scaling laws reveal *which* case applies before investing in full-scale training!

4.3 Sensitivity to Architectural Changes

Granularity of Scaling Laws

Question: How sensitive are scaling laws to small architectural differences?

Answer: Not very sensitive. Small changes (e.g., activation functions, layer normalization variants) typically:

- Affect the intercept slightly
- Have negligible impact on the slope
- Matter less than data quality or scale

Implication: For academic research proposing new activations or minor architectural tweaks:

- Must demonstrate improvements hold at scale via scaling laws
- Small improvements at one scale often disappear with more training
- "Just train longer" often outperforms minor architectural innovations

What Really Matters

Based on scaling law analysis:

Data Quality > Scale > Major Architecture > Minor Architecture

- **Data quality:** Changes slope dramatically
- **Scale:** Guaranteed improvement (if resources available)
- **Major architecture:** Transformers vs. LSTMs matters
- **Minor architecture:** Activation functions, minor tweaks—minimal impact

5 Optimal Resource Allocation: Chinchilla

5.1 The Fundamental Trade-off

Resource Allocation Problem

Given: Fixed computational budget (e.g., \$10M, 100,000 GPU-hours)

Question: Should you:

- Train a **larger model** on **less data**?
- Train a **smaller model** on **more data**?

Both consume the same compute ($\text{FLOPs} \approx 6 \times N \times D$), but which achieves lower loss?

5.2 The Chinchilla Methodology

IsoFLOP Analysis

An **IsoFLOP curve** is a set of (model size, data size) pairs that all require the same total compute:

$$\text{FLOPs} = 6 \times N \times D = \text{constant}$$

By training multiple models on the same IsoFLOP curve (varying N and D inversely), we can find the optimal trade-off.

5.2.1 Experimental Design

1. **Select compute budgets:** e.g., $10^{18}, 10^{19}, 10^{20}, 10^{21}, 10^{22}$ FLOPs
2. **For each budget, vary N and D :**
 - Example for 10^{21} FLOPs:
 - $N = 10^7$ params, $D \approx 1.67 \times 10^{13}$ tokens
 - $N = 10^8$ params, $D \approx 1.67 \times 10^{12}$ tokens
 - $N = 10^9$ params, $D \approx 1.67 \times 10^{11}$ tokens
3. **Train each configuration and measure test loss**
4. **Find optimal N for each FLOP budget**
5. **Fit scaling law relating optimal N to FLOPs**

5.3 Visual Representation

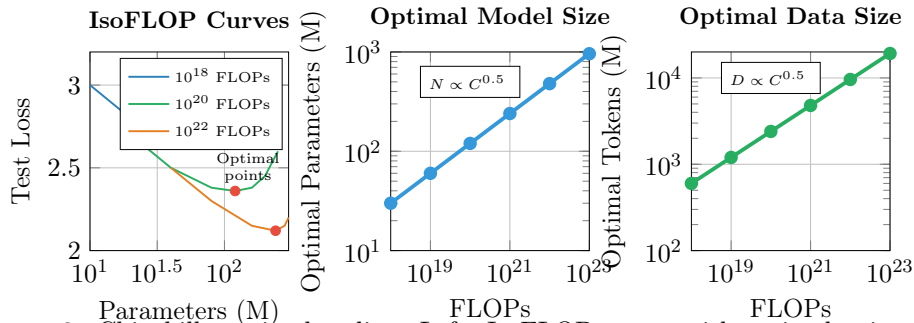


Figure 2: Chinchilla optimal scaling. Left: IsoFLOP curves with optimal points marked. Middle: Optimal parameters scale as $\sqrt{\text{FLOPs}}$. Right: Optimal tokens scale as $\sqrt{\text{FLOPs}}$.

5.4 Key Findings

Chinchilla Result: 20:1 Ratio

For training-optimal performance:

$$\text{Tokens} = 20 \times \text{Parameters}$$

For every parameter in your model, train on 20 tokens.

Examples:

- 1B parameter model $\Rightarrow$ 20B tokens

- 10B parameter model $\Rightarrow$ 200B tokens
- 100B parameter model $\Rightarrow$ 2T tokens

Training-Optimal vs. Inference-Optimal

The 20:1 ratio is **training-optimal**—it minimizes loss for a fixed training budget. However, **in production**, you must consider inference cost:

- Smaller models have lower inference costs
- Models serve millions of requests over time
- Total cost = Training cost + Lifetime inference cost

Inference-optimal ratio: Approximately **150:1** tokens per parameter

- Overtrain smaller models to reduce inference costs
- Modern production models (LLaMA 3, GPT-4) use $\sim$ 40-150:1

5.5 Mathematical Relationships

Chinchilla Scaling Laws

Given a compute budget C (in FLOPs), the optimal allocation is:

$$N_{\text{opt}} \propto C^{0.50} \quad (\text{Parameters scale as } \sqrt{C}) \quad (4)$$

$$D_{\text{opt}} \propto C^{0.50} \quad (\text{Tokens scale as } \sqrt{C}) \quad (5)$$

$$\frac{D_{\text{opt}}}{N_{\text{opt}}} \approx 20 \quad (\text{Constant ratio}) \quad (6)$$

This implies: **Double compute $\Rightarrow$ Increase both N and D by $\sqrt{2} \approx 1.41 \times$**

5.6 Implications for Model Training

Many Models Were Undertrained!

Before Chinchilla (2022), many models were trained with **insufficient data**:

Model	Parameters	Tokens	Ratio
GPT-3	175B	300B	1.7:1
Gopher	280B	300B	1.1:1
Chinchilla	70B	1.4T	20:1

Key finding: Chinchilla (70B parameters) outperformed Gopher (280B parameters) by training on more data with a smaller model!

Modern Practices

Post-Chinchilla, model developers:

- Use 40-150:1 ratios (balancing training and inference)
- Carefully optimize the parameter-data trade-off
- Recognize that "bigger isn't always better" without sufficient data

6 Broader Applications of Scaling Laws

6.1 Questions Answerable with Scaling Laws

Resource Allocation

- Train models longer vs. train bigger models?
- Collect more data vs. get more GPUs?
- Optimal budget allocation across training stages?

Data Decisions

- Data repetition / multiple epochs?
- Data mixture weighting (domains)?
- Quality vs. quantity trade-offs?

Algorithm Choices

- Architecture: LSTMs vs. Transformers?
- Model shape: Width vs. depth?
- Activation functions, normalization?

Central Principle: Test at small scale, fit scaling law, extrapolate

Overall Strategy

- When to stop training?
- ROI of additional compute?
- Predict future model performance?

6.2 Data-Related Decisions

6.2.1 Data Repetition and Multiple Epochs

Current Practice: No Epochs

Modern large-scale LLM training typically uses **no epochs** (i.e., sees each token only once):

Reasons:

- Sufficient unique data still available (for now)
- Scaling laws suggest new data is more valuable than repeated data
- Prevents memorization and overfitting

Exception: High-quality data at end of training (Wikipedia, curated sets) may be seen multiple times

6.2.2 Data Mixture Weighting

Optimizing Domain Mix

Scaling laws help determine optimal weighting across domains:

Process:

1. Train small models on different domain mixtures
2. Measure performance on target tasks
3. Fit scaling laws for each mixture
4. Extrapolate to predict large-scale performance
5. Choose mixture with best predicted performance

Common findings:

- Code data: Upweight significantly (improves reasoning)
- Books: Upweight moderately (improves coherence)
- Scientific papers: Moderate weight (domain knowledge)
- Social media: Downweight (quality concerns)

6.3 Algorithmic Decisions

6.3.1 Width vs. Depth

Model Shape Optimization

Given N parameters, should you build:

- **Wide and shallow:** Fewer layers, larger hidden dimensions
- **Narrow and deep:** More layers, smaller hidden dimensions

Scaling law approach:

1. Train models at various widths/depths keeping N constant
2. Measure performance across scales
3. Identify which configuration scales better

General finding: Depth scales better than width, but optimal ratio depends on total size

6.4 Strategic Planning

6.4.1 Predicting Future Performance

Long-term Forecasting

Scaling laws enable **multi-year performance predictions**:

- Given current hardware trends (Moore's Law variants)
- Estimate available compute in 2-3 years
- Apply scaling laws to predict achievable loss
- Translate to expected capabilities

Example: In 2020, scaling laws suggested GPT-4-level performance would be achievable by 2023-2024 with sufficient compute—which proved accurate!

Limitations of Forecasting

Predictions assume:

- Scaling laws continue to hold (no plateaus)
- Sufficient high-quality data remains available
- No fundamental algorithmic breakthroughs (or setbacks)
- Infrastructure scales as expected

Any of these assumptions breaking could invalidate predictions.

7 The Bitter Lesson

7.1 Richard Sutton's Insight

The Bitter Lesson

In 2019, reinforcement learning pioneer Richard Sutton wrote a famous blog post titled "*The Bitter Lesson*", arguing:

"The biggest lesson that can be read from 70 years of AI research is that general methods that leverage computation are ultimately the most effective."

Core thesis: Attempts to build domain knowledge into AI systems are eventually surpassed by simple methods that scale with computation.

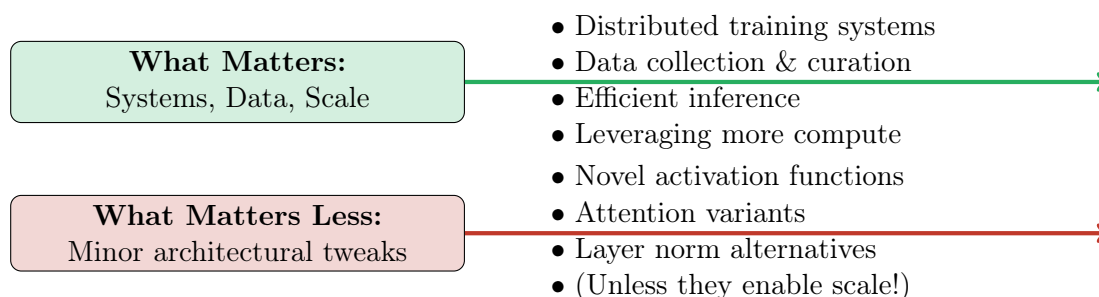
7.2 Application to LLMs

Scaling Laws Validate the Bitter Lesson

LLM scaling laws provide strong empirical support:

1. **Compute scales predictably:** Moore's Law ensures exponentially growing compute
2. **Performance scales with compute:** Scaling laws guarantee improvement with more resources
3. **Conclusion:** The *only* thing that matters long-term is having architectures and systems that can **leverage computation efficiently**

7.3 Implications for Research



Research Priorities

High impact:

- Better data collection and filtering methods
- More efficient training systems (lower cost per FLOP)
- Improved inference efficiency (serve more requests)
- Architectural changes that enable larger scale

Lower impact (in the long run):

- Minor modifications to existing architectures
- Clever tricks that don't change scaling properties
- Methods that work well at small scale but don't scale

The Researcher's Dilemma

The "bitter" part: Many researchers (including the lecturer) spent years on clever architectural innovations, only to see them superseded by simple methods at larger scale.
Lesson: Complexity is often the enemy of scale. Simple methods that scale well beat complex methods that don't.

7.4 Philosophical Implications

Deep Truth

Scaling laws reveal something profound about intelligence and learning:

- **Quantity over quality (of algorithms):** More data + compute often beats better algorithms
- **Emergence over engineering:** Capabilities emerge from scale rather than explicit design
- **Simplicity over sophistication:** Simple architectures that scale beat complex ones that don't

This is "*bitter*" because it suggests much of traditional AI research focused on the wrong things—but it's also *empowering* because it provides a clear path forward.

The OpenAI Philosophy

OpenAI's success with GPT series exemplifies the bitter lesson:

- **GPT-1 (2018):** Demonstrated feasibility
- **GPT-2 (2019):** Scaled up (1.5B parameters)
- **GPT-3 (2020):** Scaled up more (175B parameters)
- **GPT-4 (2023):** Scaled up further (estimated >1T parameters)

Key insight: Success came not from architectural innovation, but from:

1. Believing scaling would work
2. Building systems to enable scale
3. Executing at increasingly large scales

8 Real-World Cost Analysis: LLaMA 3 400B

8.1 Model Specifications

LLaMA 3 400B: By the Numbers

- **Parameters:** 405 billion (405B)
- **Training Data:** 15.6 trillion tokens (15.6T)
- **Token-to-Parameter Ratio:** $\frac{15.6T}{405B} \approx 38.5$ tokens/param
- **Hardware:** 16,000 NVIDIA H100 GPUs
- **Throughput:** 400 TFLOPS per H100 (average)
- **Release:** July 2024
- **Status:** Best open-source model (as of August 2024)

Training vs. Inference Optimal

LLaMA 3's 38.5:1 ratio sits between:

- Chinchilla training-optimal: 20:1
- Inference-optimal: $\sim 150:1$

This suggests Meta optimized for **training cost** while maintaining reasonable inference costs.

8.2 FLOP Calculation

Training FLOPs Formula

For training a Transformer language model:

$$\text{FLOPs} \approx 6 \times N \times D$$

where:

- N = number of parameters
- D = number of training tokens
- Factor of 6 accounts for forward pass ($2\times$), backward pass ($4\times$)

Note: This is an approximation. Exact value depends on implementation details, but 6 is standard.

8.2.1 LLaMA 3 Computation

$$\begin{aligned}
 \text{FLOPs} &= 6 \times N \times D \\
 &= 6 \times 405 \times 10^9 \times 15.6 \times 10^{12} \\
 &= 6 \times 405 \times 15.6 \times 10^{21} \\
 &= 6 \times 6318 \times 10^{21} \\
 &= 37,908 \times 10^{21} \\
 &= \boxed{3.79 \times 10^{25} \text{ FLOPs}}
 \end{aligned}$$

Executive Order Threshold

The Biden administration’s Executive Order on AI (2023) requires special scrutiny for models trained with:

$$\text{FLOPs} \geq 10^{26}$$

LLaMA 3: 3.79×10^{25} FLOPs $\approx$ **2.6× below threshold**

This was likely *intentional*—Meta stopped training just under the regulatory limit!

8.3 Time and Compute Requirements

8.3.1 GPU-Hours Calculation

$$\begin{aligned}
 \text{GPU-hours} &= \frac{\text{Total FLOPs}}{\text{FLOPS per GPU} \times 3600 \text{ s/hour}} \\
 &= \frac{3.79 \times 10^{25}}{400 \times 10^{12} \times 3600} \\
 &= \frac{3.79 \times 10^{25}}{1.44 \times 10^{18}} \\
 &= 2.63 \times 10^7 \text{ GPU-hours} \\
 &= \boxed{26.3 \text{ million GPU-hours}}
 \end{aligned}$$

Meta’s Reported Number

Meta reported approximately **30 million GPU-hours** in their technical report. The 3.7M hour discrepancy likely due to:

- Training instabilities and restarts
- Checkpoint overhead
- Non-optimal utilization
- Experimentation and hyperparameter tuning

This suggests $\sim 88\%$ efficiency, which is actually quite good for such a large-scale training run!

8.3.2 Wall-Clock Time

With 16,000 H100 GPUs running simultaneously:

$$\begin{aligned}\text{Days} &= \frac{26.3 \times 10^6 \text{ GPU-hours}}{16,000 \text{ GPUs} \times 24 \text{ hours/day}} \\ &= \frac{26.3 \times 10^6}{384,000} \\ &= \boxed{68.5 \text{ days} \approx 70 \text{ days}}\end{aligned}$$

Over two months of continuous training on 16,000 top-tier GPUs!

8.4 Cost Breakdown

8.4.1 Compute Costs

GPU Rental Pricing

NVIDIA H100 rental cost: Lower bound $\approx$ \$2 per GPU-hour

Note: This is the minimum market rate. Actual costs may be higher depending on:

- Provider (AWS, Azure, GCP, specialized providers)
- Contract terms (spot vs. reserved instances)
- Region and availability
- Network and storage requirements

$$\begin{aligned}\text{Compute cost} &= \text{GPU-hours} \times \text{Cost per GPU-hour} \\ &= 26.3 \times 10^6 \times \$2 \\ &= \boxed{\$52.6 \text{ million}}\end{aligned}$$

Meta's Actual Cost

Meta owns its infrastructure, so doesn't pay rental rates. However, **opportunity cost** is similar:

- GPU depreciation
- Power consumption (significant!)
- Datacenter costs
- Maintenance and operations

Reasonable estimate: \$40-60M in direct costs for compute

8.4.2 Personnel Costs

Personnel = Number of employees $\times$ Avg. compensation $\times$ Time fraction

$$\approx 50 \text{ employees} \times \$500,000/\text{year} \times \frac{70}{365}$$

$$= \$25,000,000 \times 0.192$$

$$\approx \boxed{\$4.8 \text{ million}}$$

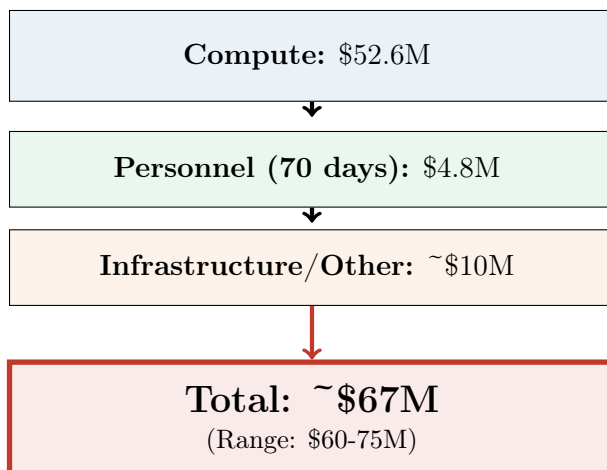
Team Composition

Typical large-scale LLM project team (~ 50 people):

- **Data team** (~ 15): Collection, filtering, preprocessing
- **Systems/infrastructure** (~ 10): Distributed training, optimization
- **Modeling** (~ 10): Architecture, hyperparameters, experiments
- **Evaluation** (~ 5): Benchmarking, analysis
- **Safety/alignment** (~ 5): Red-teaming, alignment
- **Management/coordination** (~ 5): Project management, coordination

Compensation: \$500K/year is reasonable for senior engineers at top companies (salary + equity + benefits)

8.4.3 Total Cost



Cost Breakdown

Category	Cost	Percentage
Compute (GPUs)	\$52.6M	78%
Personnel (70 days)	\$4.8M	7%
Infrastructure/other	\$10M	15%
Total	\$67M	100%

Key insight: Compute dominates costs. Even highly-paid engineers are a small fraction of total expense!

8.5 Environmental Impact

8.5.1 Carbon Emissions Calculation

$$\begin{aligned}
 \text{Energy consumption} &= \text{GPU-hours} \times \text{Power per GPU} \\
 &= 26.3 \times 10^6 \text{ hours} \times 0.7 \text{ kW} \\
 &= \boxed{18.41 \text{ million kWh}}
 \end{aligned}$$

$$\begin{aligned}
 \text{CO}_2 \text{ emissions} &= \text{Energy} \times \text{Carbon intensity} \\
 &= 18.41 \times 10^6 \text{ kWh} \times 0.24 \text{ kg CO}_2/\text{kWh} \\
 &= \boxed{4,418 \text{ tons CO}_2\text{-equivalent}}
 \end{aligned}$$

Context: Carbon Footprint

How significant is 4,418 tons of CO₂?

Comparison:

- **JFK to London return ticket:** ~2 tons CO₂ per passenger
- **4,418 tons CO₂ ≈ 2,209 return tickets**
- Average American emits ~16 tons CO₂/year
- 4,418 tons ≈ 276 Americans' annual emissions

Perspective:

- Significant in absolute terms
- Relatively small compared to global AI/tech sector
- May become a critical issue if training costs increase 100× (GPT-6, GPT-7)
- Current environmental impact is *manageable* but *growing rapidly*

Future Concerns

If model training scales 100× in the next few generations:

- Energy: 1.8 *billion* kWh per model
- Carbon: 440,000 tons CO₂ per model
- This would become a **serious environmental concern**

Mitigation strategies:

- Use renewable energy for datacenters
- Improve training efficiency

- Better model reuse and transfer learning
- Optimize inference to reduce total lifecycle impact

8.6 The Next Generation

Scaling Trends

Historical pattern: Each generation multiplies compute by $\sim 10\times$

Model	Year	Parameters	FLOPs
GPT-2	2019	1.5B	$\sim 10^{22}$
GPT-3	2020	175B	$\sim 10^{23}$
GPT-4	2023	$\sim 1\text{T}$ (est.)	$\sim 10^{25}$
LLaMA 3	2024	405B	3.8×10^{25}
GPT-5?	2025-26	?	$\sim 10^{26}$
GPT-6?	2027-28	?	$\sim 10^{27}$

If trend continues:

- **Next model (2025-26):** $\sim \$500\text{M}-\1B
- **Two generations out (2027-28):** $\sim \$5\text{B}-\10B

Constraints on Continued Scaling

What could limit $10\times$ scaling per generation?

1. **Data scarcity:** Running out of high-quality text
2. **Energy/power:** Datacenter power requirements
3. **Cost:** $\$10\text{B}$ training runs may not be economically viable
4. **Chips:** Manufacturing enough H100s/H200s/B200s
5. **Plateauing:** Scaling laws may eventually break down

Most experts believe we have 1-2 more generations of $10\times$ scaling before hitting serious constraints.

9 Summary and Key Takeaways

9.1 Core Principles

The Five Pillars of Scaling Laws

1. **Power-law relationships:** Performance scales predictably with compute, data, and parameters
2. **Log-log linearity:** Makes forecasting and extrapolation tractable
3. **Training vs. inference:** Optimal ratios differ (20:1 vs. 150:1)

4. **Architectural comparisons:** Scaling laws reveal which architectures are better long-term
5. **The Bitter Lesson:** Simple methods that scale beat complex methods that don't

9.2 Practical Guidelines

For Practitioners

When training LLMs:

- **Start small:** Validate approaches at 100M-1B scale before committing to large training runs
- **Fit scaling laws:** Train multiple model sizes and extrapolate
- **Respect the ratios:** Use 40-150 tokens per parameter
- **Focus on data:** Data quality has more impact than minor architectural changes
- **Plan for inference:** Production costs are dominated by serving, not training

9.3 Strategic Insights

1. **Scaling is predictable:** We can forecast performance years in advance

2. **Size matters differently:** Optimal size depends on whether you optimize for training or inference

3. **Architecture is secondary:** Transformers won, but minor tweaks don't matter much

4. **Data is king:** Quality and quantity of data determine the slope of your scaling curve

5. **Constraints loom:** Data scarcity and energy costs may limit future scaling

9.4 Open Questions

Unresolved Issues

1. **When will scaling laws plateau?** No current evidence, but likely eventually
2. **How to handle data scarcity?** Synthetic data? Multimodal? Better curation?
3. **Energy and sustainability:** Can we maintain 10× scaling per generation?
4. **Emergent capabilities:** Can we predict what capabilities emerge at what scale?
5. **Beyond next-token prediction:** Will fundamentally different objectives change scaling properties?

9.5 Looking Forward

The Road Ahead

Near-term (1-2 years):

- Continued 10× scaling per generation
- Models approaching 10^{26} FLOPs
- Training costs: \$500M-\$1B
- Scaling laws continue to hold

Medium-term (3-5 years):

- Potential data scarcity bottleneck
- Synthetic data becomes critical
- Energy constraints may limit growth
- Possible saturation of scaling laws

Long-term (5+ years):

- New paradigms beyond pure scaling?
- Multimodal and embodied models
- Focus shifts from pretraining to post-training?
- Efficiency and inference optimization dominate

A Mathematical Reference

A.1 Key Formulas

A.1.1 Scaling Law Power Laws

$$L_{\text{compute}} = \left(\frac{C_{\text{crit}}}{C} \right)^{\alpha_C} \times 10^{\beta_C + \epsilon_C^2} \quad (7)$$

$$L_{\text{data}} = \left(\frac{D_0}{D} \right)^{\alpha_D} \times 10^{\beta_D + \epsilon_D^2} \quad (8)$$

$$L_{\text{params}} = \left(\frac{N_0}{N} \right)^{\alpha_N} \times 10^{\beta_N + \epsilon_N^2} \quad (9)$$

A.1.2 Chinchilla Optimal Scaling

$$N_{\text{opt}} \propto C^{0.5} \quad (10)$$

$$D_{\text{opt}} \propto C^{0.5} \quad (11)$$

$$\frac{D_{\text{opt}}}{N_{\text{opt}}} \approx 20 \text{ (training-optimal)} \quad (12)$$

$$\frac{D_{\text{opt}}}{N_{\text{opt}}} \approx 150 \text{ (inference-optimal)} \quad (13)$$

A.1.3 FLOP Calculation

$$\text{Training FLOPs} = 6 \times N \times D \quad (14)$$

$$\text{where } N = \text{parameters, } D = \text{tokens} \quad (15)$$

A.1.4 Time and Cost

$$\text{GPU-hours} = \frac{\text{FLOPs}}{\text{FLOPS per GPU} \times 3600} \quad (16)$$

$$\text{Days} = \frac{\text{GPU-hours}}{\text{Number of GPUs} \times 24} \quad (17)$$

$$\text{Cost} = \text{GPU-hours} \times \text{Cost per GPU-hour} \quad (18)$$

A.2 Notation Guide

Symbol	Meaning
L	Test loss (cross-entropy)
C	Compute budget (FLOPs or PF-days)
N	Number of parameters (non-embedding)
D	Dataset size (number of tokens)
α	Scaling exponent (power-law slope)
β	Intercept term in power law
ϵ	Residual error
PPL	Perplexity (2^L)
FLOPS	Floating point operations per second
FLOPs	Total floating point operations
PF-days	PetaFLOP-days (10^{15} FLOPS $\times$ days)

B Additional Resources

B.1 Seminal Papers

- **Kaplan et al. (2020):** "Scaling Laws for Neural Language Models" (OpenAI)
- **Hoffmann et al. (2022):** "Training Compute-Optimal Large Language Models" (Chinchilla, DeepMind)
- **Sutton (2019):** "The Bitter Lesson" (Blog post)
- **Hestness et al. (2017):** "Deep Learning Scaling is Predictable, Empirically"

B.2 Technical Reports

- **LLaMA 3:** Meta AI Technical Report (2024)
- **GPT-4:** OpenAI Technical Report (2023)
- **PaLM:** Google Research (2022)

B.3 Online Resources

- Richard Sutton's "Bitter Lesson": <http://www.incompleteideas.net/IncIdeas/BitterLesson.html>
- Epoch AI Scaling Laws Analysis: <https://epochai.org/>
- Papers With Code - Scaling Laws: <https://paperswithcode.com/>

C Glossary

Compute Budget

Total computational resources (measured in FLOPs) allocated for training.

IsoFLOP Curve

Set of (parameters, data) configurations requiring identical compute.

Power Law

Mathematical relationship of the form $y = ax^b$ (linear in log-log space).

Scaling Rate

The slope (exponent) in a power-law relationship; determines how quickly performance improves with scale.

Training-Optimal

Configuration that minimizes loss for a given training budget.

Inference-Optimal

Configuration that minimizes total cost (training + lifetime inference).

The Bitter Lesson

The observation that simple methods leveraging computation ultimately beat complex hand-crafted approaches.

Chinchilla Scaling

The finding that optimal token-to-parameter ratio is approximately 20:1 for training efficiency.